ParseViz Project Documentation

ParseViz is a web studio that shows every step of LL(1) parsing for any grammar: transformation, FIRST/FOLLOW, table, trace and an animated parse tree.

1. Problem statement

Context-free grammars, left recursion, left factoring, FIRST/FOLLOW sets and LL(1) tables are among the most abstract topics in compiler design (PS-03, Unit II: Top-Down Parsing). Students often memorise the elimination steps without understanding why they are needed, and they never see the intermediate work behind a table cell or a rejected string.

The task is to build a grammar practice studio where a student can:

1. Enter a grammar and choose a valid, invalid or empty test string.
2. See immediate left recursion removed, step by step.
3. See common prefixes found and the grammar left-factored.
4. Compute FIRST and FOLLOW sets, with the productions that contribute to each.
5. Generate the LL(1) table, highlight empty entries and report conflicts.
6. Trace the parse with stack, remaining input and action at every step.
7. See the parse tree for accepted strings and a clear error location for rejected ones.

Bonus: a practice mode that generates new strings and checks the student's accept/reject predictions.

2. Proposed solution

ParseViz is a Flask web application. The student types a grammar and a test string; the Python engine runs the whole top-down pipeline and the browser shows each stage in its own tab.

Tab	What the student sees
1. Transformations	Original and final grammar; every left-recursion step (alpha, beta, new non-terminal), indirect recursion by substitution; left factoring by longest common prefix, repeated until none remains
2. FIRST / FOLLOW	A log of every set addition with its pass, production and rule; hover a symbol to see which production put it there
3. LL(1) Table	Filled cells, empty cells shown as "-", conflicts highlighted and typed as FIRST/FIRST, FIRST/FOLLOW or FOLLOW/FOLLOW
4. Parsing Trace	Stack, remaining input and action per step; the error token is marked with its position and the expected symbols
5. Parse Tree	Animated D3 tree driven by the trace (play, pause, step, slider); partial tree with a red node for rejected input

Tab	What the student sees
6. Practice	New strings are generated, the student predicts accept or reject, and the real parser gives the verdict, score and streak

Extras: one-click sample strings (valid, invalid, empty), input like id+id*id without spaces is split into tokens, seven built-in example grammars, a bounded ambiguity check, and dark/light themes. On desktop screens the Grammar Setup panel stays fixed and the tab bar stays pinned while the results scroll, so the grammar and test string are always in view.

3. Studio walkthrough

The screenshots below were captured from the running application (Flask development server, 1440 × 900 browser window) using the built-in example grammars. Each figure shows one stage of the pipeline.

3.1 Overview and layout

The Grammar Setup panel (example grammars, grammar editor, test string and quick-test chips) is on the left. The right side shows the status chips and the six tabs. Figure 1 shows the Transformations tab for the expression grammar.

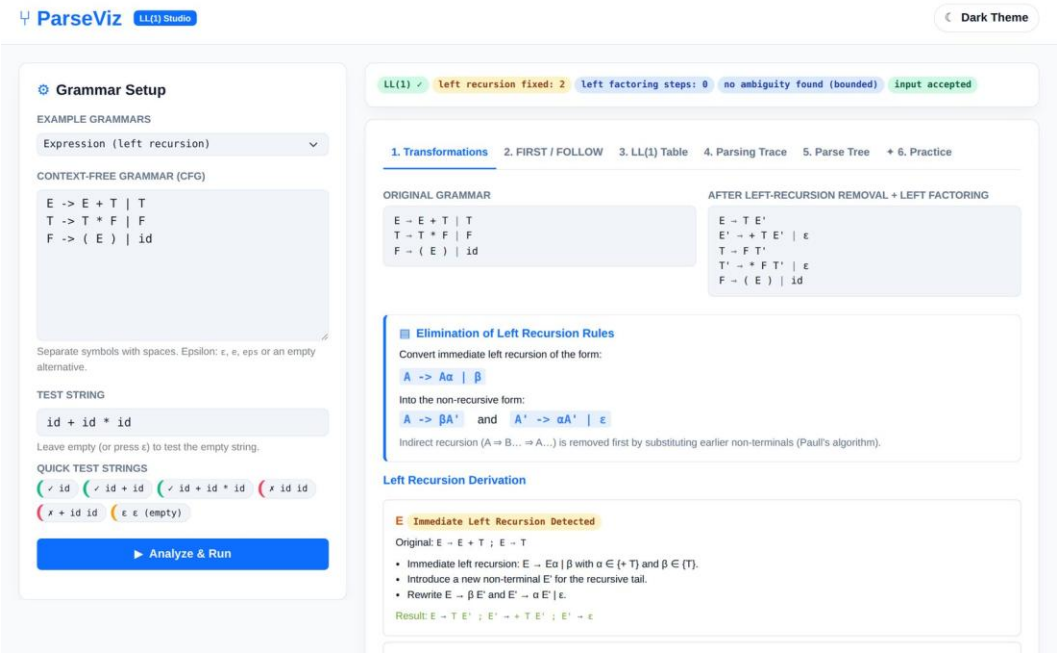


*Figure 1. Studio overview: expression grammar E → E + T | T, T → T * F | F, F → (E) | id, with the original and transformed grammar.*

On desktop screens the setup panel stays fixed and the tab bar stays pinned while the results scroll (Figure 2).

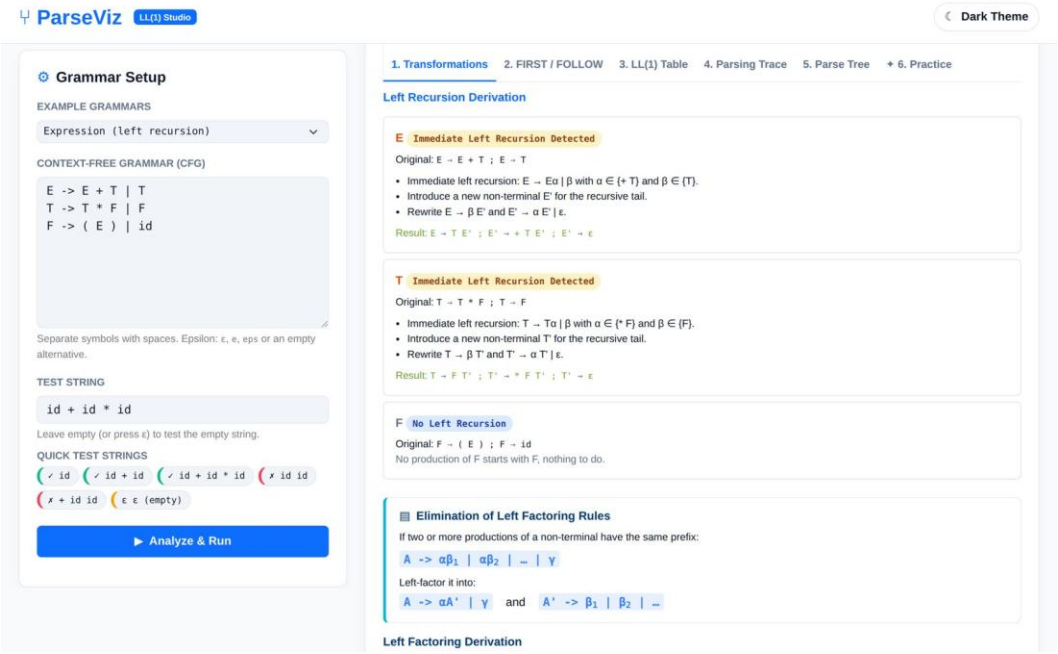


Figure 2. Page scrolled down: the Grammar Setup panel and the tab bar stay in view while the left-recursion derivation scrolls.

3.2 Left recursion and left factoring

Each rewrite is listed with the rule before and after. For A → a b c | a b d | a x | y the studio finds the common prefix a, then the nested prefix b, and repeats until no prefix is left.

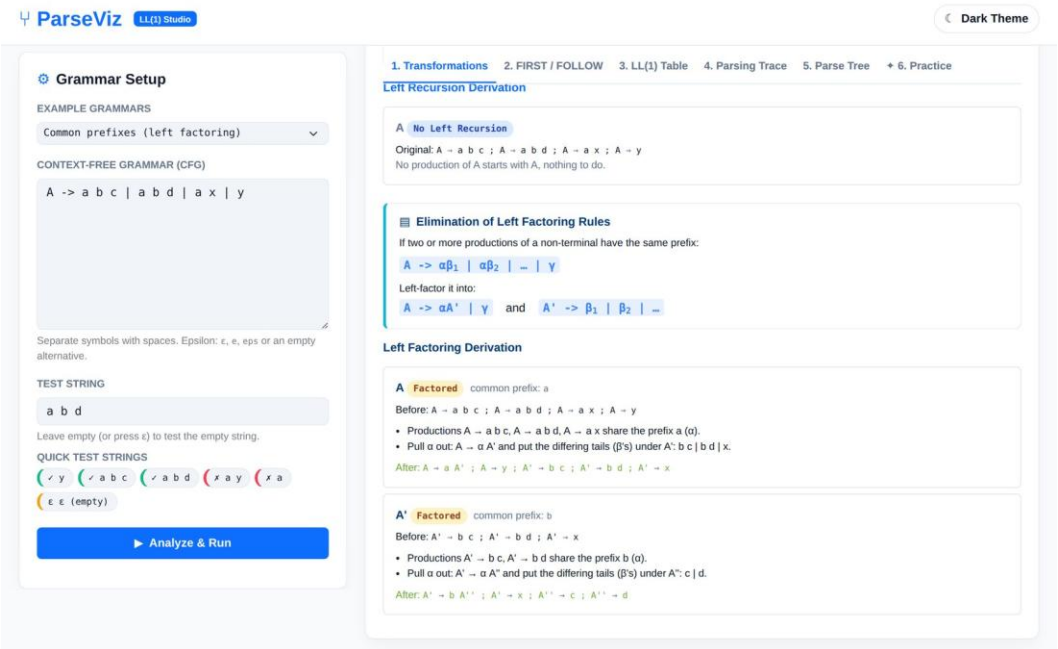


Figure 3. Left factoring derivation for A → a b c | a b d | a x | y (common prefix a, then b).

3.3 FIRST and FOLLOW

The rules come first, followed by a log with one row per set addition: the pass, the set, the symbols added and the production that caused it.

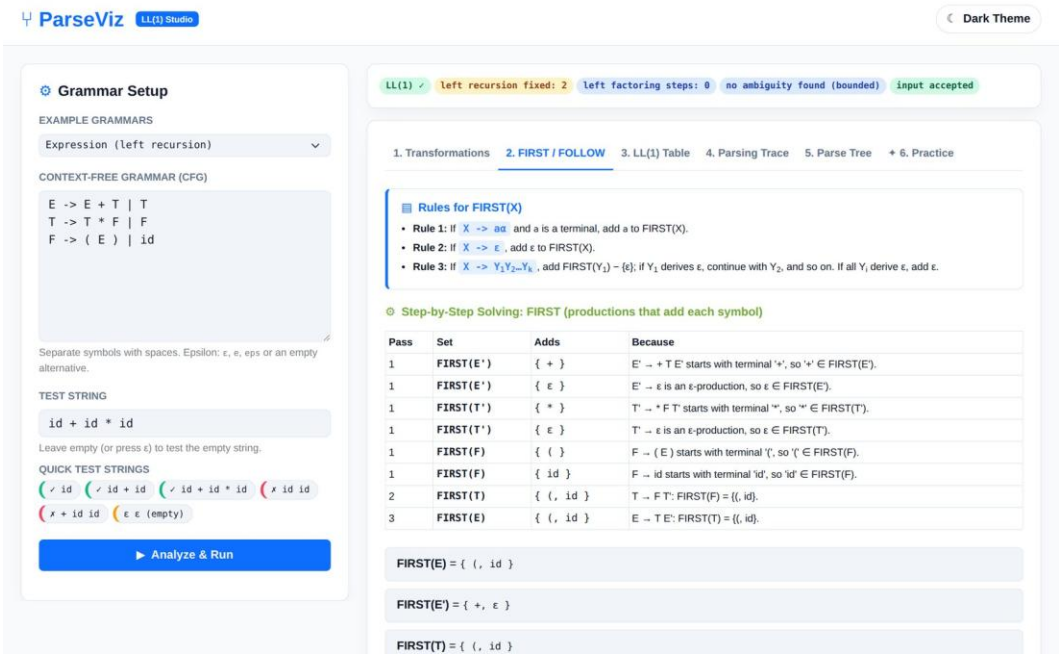


Figure 4. FIRST tab for the transformed expression grammar: rules and the step-by-step solving log.

3.4 LL(1) table and conflicts

Filled cells are shown with their production, empty cells with "-" (a syntax error if reached). Conflicts are highlighted and typed as FIRST/FIRST, FIRST/FOLLOW or FOLLOW/FOLLOW.

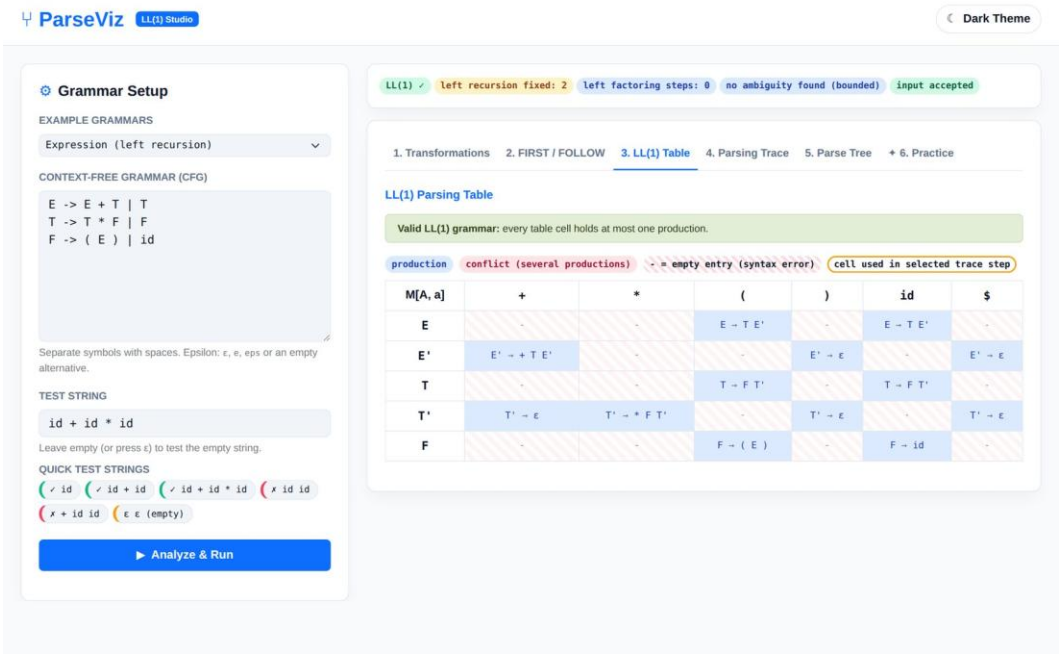


Figure 5. LL(1) parsing table for the expression grammar: the grammar is valid LL(1), every cell holds at most one production.

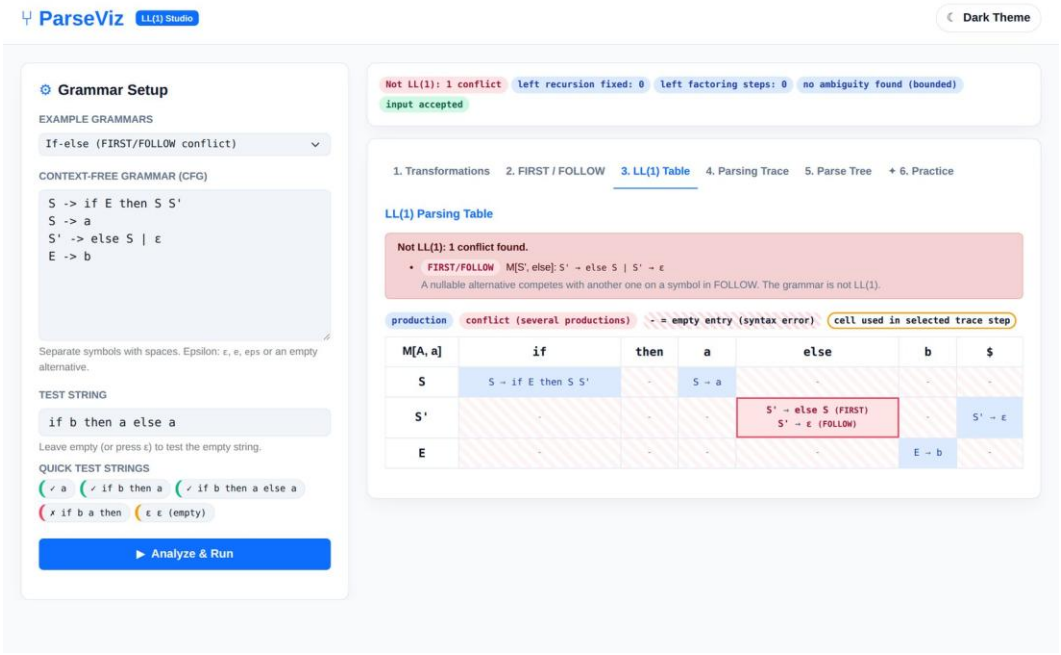


Figure 6. If-else grammar: M[S', else] holds two productions, reported as a FIRST/FOLLOW conflict and highlighted in red.

3.5 Parsing trace

The trace lists the stack, the remaining input and the action for every step. Clicking a row jumps the table and the tree to that step.

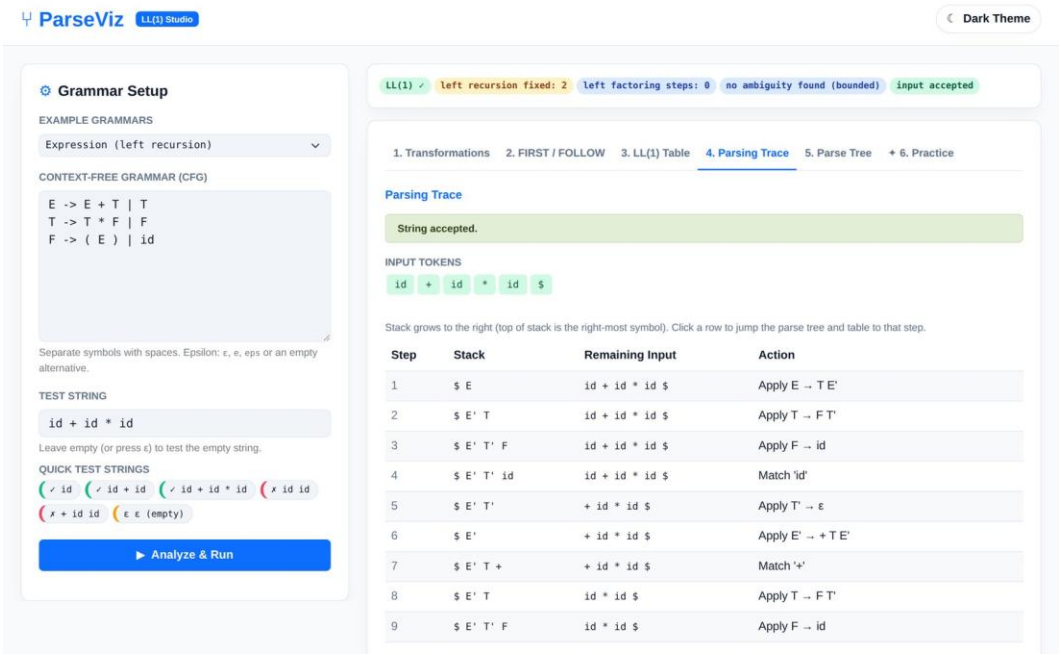


*Figure 7. Accepted input "id + id * id": stack, remaining input and action at each step.*

For rejected input the bad token is marked in the token strip, with its position and the expected symbols.

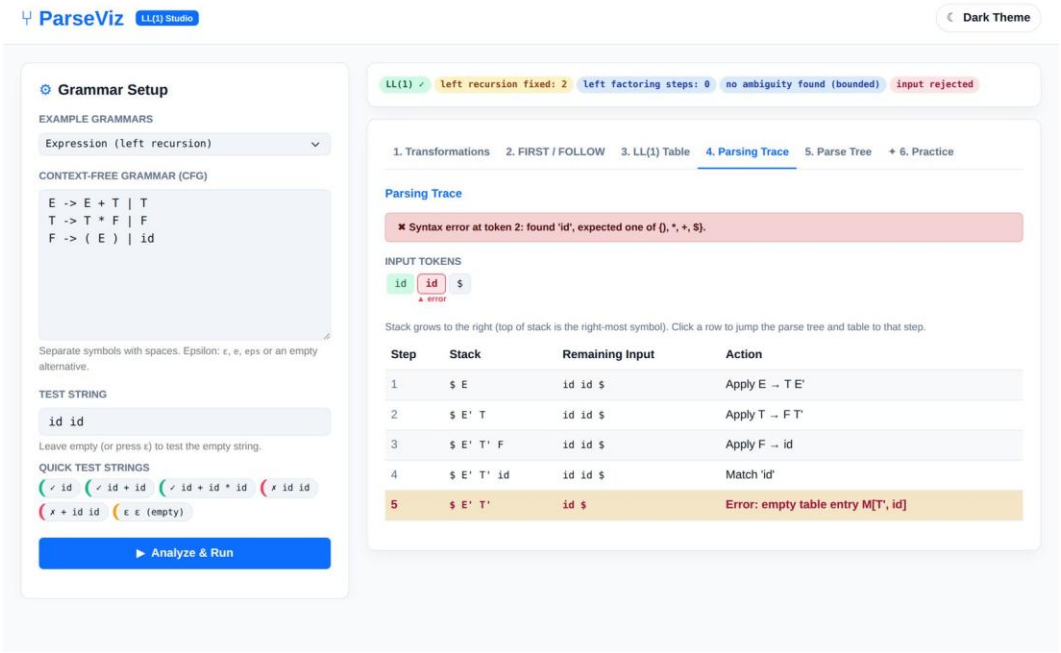


Figure 8. Rejected input "id id": syntax error at token 2, with the failing step highlighted in the trace.

3.6 Parse tree

The tree is built from the trace and can be played, paused, stepped or scrubbed with the slider. Matched terminals are green and ε leaves are dashed.

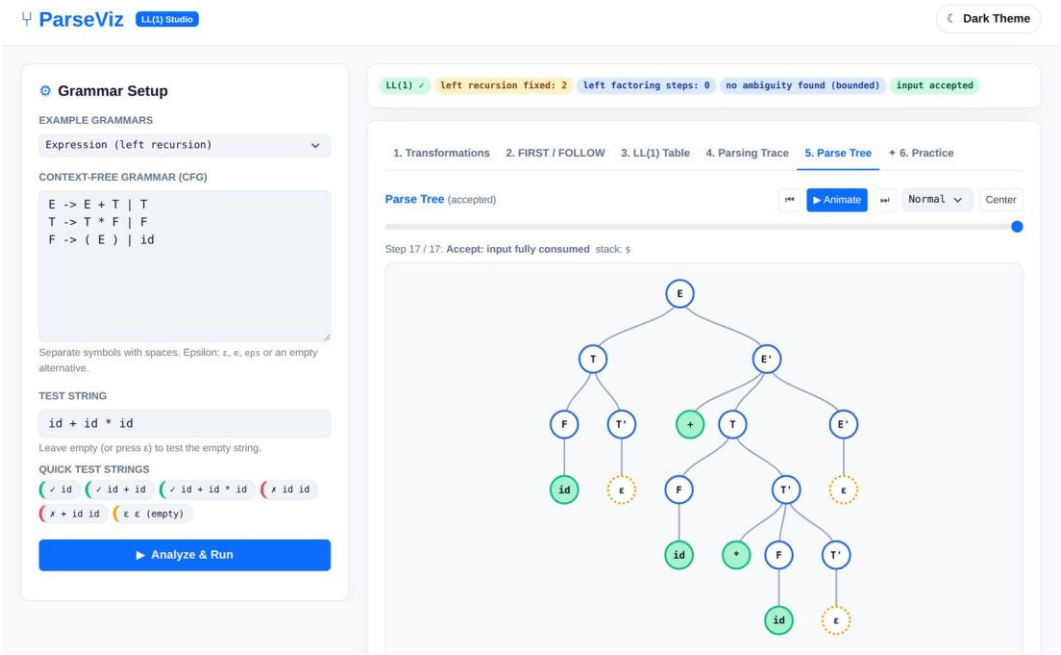


*Figure 9. Complete parse tree for "id + id * id" (step 17 of 17).*

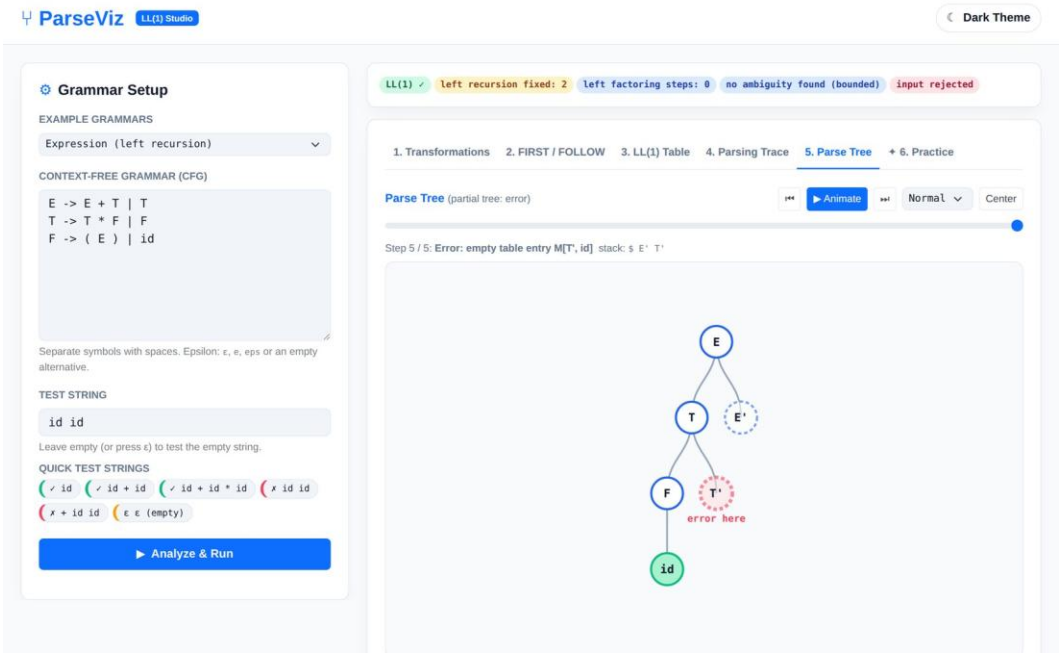


Figure 10. Partial parse tree for the rejected input "id id": the failing node is marked "error here".

3.7 Practice mode

New strings are generated from the current grammar. The student predicts accept or reject and the real parser gives the verdict, the score and the streak.

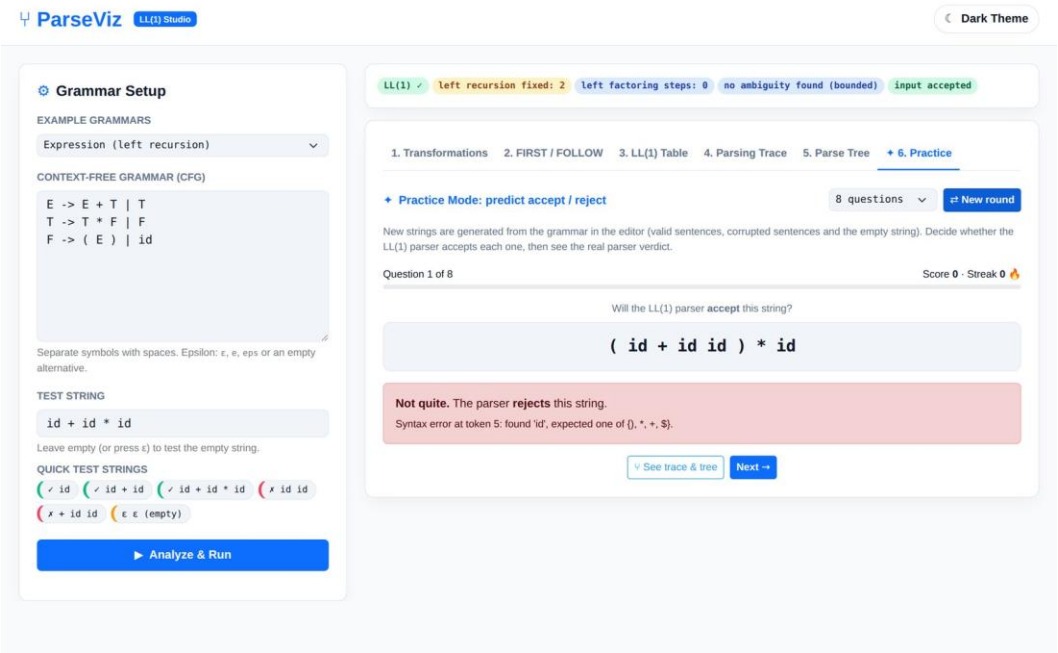


Figure 11. Practice mode after one answer: verdict, reason, score and streak.

3.8 Dark theme

A theme toggle in the header switches between light and dark.

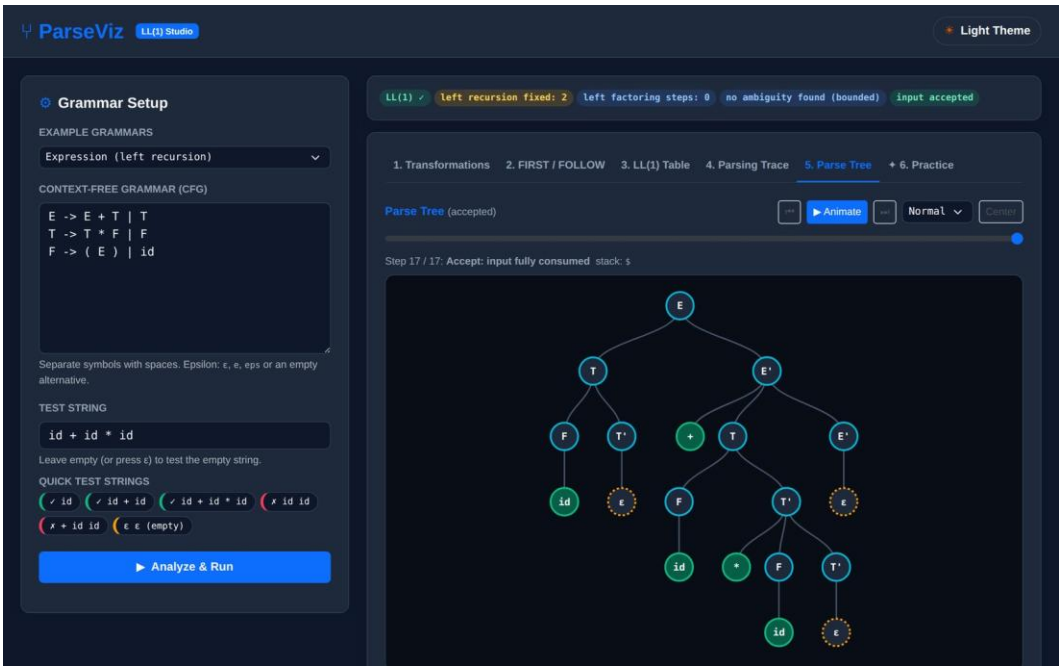


Figure 12. Parse tree in the dark theme.

4. Tools and technologies

Tool	Used for
Python 3	The parsing engine (parser_engine.py): all algorithms, no web code
Flask	Web server and JSON API (app.py)
HTML, CSS, JavaScript	The studio page (templates/index.html)
Bootstrap 5	Layout, tabs, tables and light/dark theming
D3.js v7	Animated parse tree with zoom and pan
unittest	18 engine tests for edge cases
gunicorn, Render	Deployment from GitHub
Git and GitHub	Version control

5. How to run it (Windows)

Install Python 3.10 or newer first, then open Command Prompt in the project folder.

1. Go to the folder: `cd C:\Users\Dell\Desktop\parseviz`
2. Install the requirements once: `pip install -r requirements.txt`
3. Start the server: `python app.py`
4. Open http://127.0.0.1:5000 in a browser (the page loads Bootstrap and D3 from a CDN, so it needs internet).
5. Stop the server with Ctrl+C.

To use it: type or pick a grammar, enter a test string (or leave it empty for the empty string), click Analyze & Run, then walk through tabs 1 to 5. Open tab 6 to practise.

To run the tests: `python -m unittest discover -s tests`

To deploy: push the folder to GitHub, then create a Render web service with build command `pip install -r requirements.txt` and start command `gunicorn app:app`.

6. Testing and verification

The engine is covered by 18 unit tests, run with python -m unittest discover -s tests. All 18 pass on the current code.

Area (tests)	What is checked
Left recursion (4)	Expression grammar; ε base case A → A a | ε; indirect recursion by substitution; guard against non-terminating recursion
Left factoring (1)	Longest common prefix, repeated for nested prefixes
FIRST / FOLLOW (1)	Sets for the expression grammar
LL(1) table (3)	Dangling-else conflict; several conflicts in one grammar; no false conflict for a nullable production
Parsing (4)	Rejection with error location; missing parenthesis; input without spaces; empty string with ε
Tree and practice (3)	Parse tree shape; partial tree on error; practice round
Input and ambiguity (2)	Clear message for a bad grammar; bounded ambiguity check

Besides the unit tests, every screen in section 3 was checked in a browser against the built-in example grammars (accepted, rejected and empty strings, a conflict grammar and practice mode).

7. Conclusion

ParseViz turns top-down parsing from a memorised recipe into something a student can watch: each grammar rewrite, each set entry, each table cell and each stack move is shown and explained. It covers all seven requirements of PS-03 and the practice-mode bonus, handles epsilon productions, indirect recursion, nested prefixes and multiple conflicts, and is covered by 18 automated tests.

Possible next steps: LR/bottom-up parsing views, exporting traces as PDF, and saving a student's practice history.